

SESSION 2 - EXAMEN D'ALGORITHMIQUE - SDA - L31

Exercice 1 (2.5 Pts=1.0+1.5)

- 1) En ne comptabilisant que les **multiplications (ligne I & II)** ; étudier la complexité en temps de l'algorithme tata ci-dessous :

<pre> Fonction tata(n, x, coef) → Reel Debut s ← 0 Pour i variant de 1 à n faire p ← 1 Pour j variant de 1 à i faire p ← p*x Fin Pour s ← s + p*coef[i] Fin Pour Renvoyer s Fin </pre>	Paramètres Formels : n : Entier(E) x : Reel (E) coef : tableau de reels (E) Variables intermédiaires : s, p : Reels
--	--

- 2) En ne comptabilisant que les **additions (lignes I et II)** ; trouver puis résoudre en fonction de n l'équation de la complexité de la fonction tata (Justifier votre réponse) :

<pre> Fonction tata(n, a, b) → Entier Début si n = 1 alors (I) renvoyer a + b sinon x ← tata(n-1, a, b) y ← tata(n-1, x, 2*b) (II) renvoyer x + y fin si Fin </pre>	Paramètres Formels n : entier. Paramètre en entrée a, b : entiers. Paramètre en entrée/sortie Variables intermédiaires : x, y : entier.
---	--

Exercice 2 (5.0 Pts=0.5+0.75+0.75+2.0+1.0)

Le but de cet exercice est de trier par ordre croissant un tableau composé de n éléments en passant par une liste doublement chaînée (ListeTQ) définie comme suit :

```

typedef int TElement;
typedef struct cellule{
  TElement donnee ;
  struct cellule* suivant ;
  struct cellule* precedent;
}*ListeTQ;

```

- 1) **Ecrire en C**, l'algorithme de création de la première cellule de la listeTQ à partir d'un élément donné.
- 2) **Ecrire en C**, l'algorithme d'insertion en tête d'un élément donné dans une ListeTQ non vide donnée.
- 3) **Ecrire en C**, l'algorithme d'insertion en queue d'un élément donné dans une ListeTQ non vide donnée.
- 4) En parcourant la ListeTQ **triée** dans les deux sens ; écrire en langage algorithmique (ou C), l'algorithme d'insertion triée d'un élément donné dans une ListeTQ donnée.

- 5) En passant par la ListeTQ comme structure intermédiaire ; écrire en langage algorithmique ou en C ; l'algorithme de tri d'un tableau donné composé de n éléments.

Note : on suppose connu l'algorithme de suppression d'une ListeTQ.

Exercice 3 (6.5 Pts=1.0+1.5+1.0+0.75+1.0+0.75+0.5)

Note : on suppose connu l'algorithme d'équilibrage.

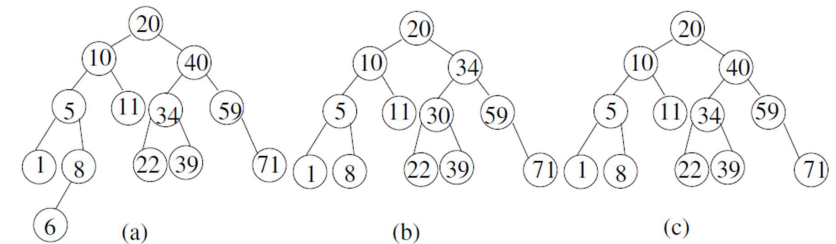
On représente un AVL par la structure de données suivante :

```

typedef struct noeud{
  int hauteur;
  TElement donnee;
  struct noeud *filsG, *filsD ;
} *AVL;

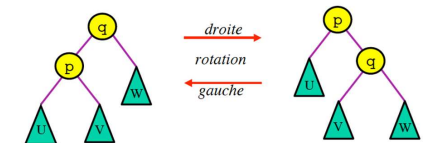
```

- 1) Les arbres ci-dessous sont-ils des AVL ? (Justifier votre réponse).



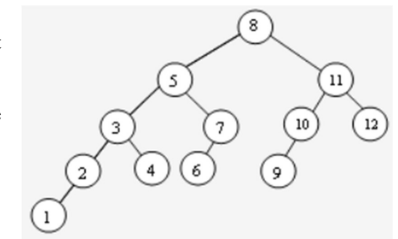
- 2) Ecrire en C, les algorithmes de rotations suivants :

- Rotation simple Gauche.
- Rotation simple droite.
- Rotation double gauche-droite.
- Rotation double droite-gauche.



- 3) **Ecrire en C**, l'algorithme d'insertion d'un élément donné dans un AVL donné.
- 4) Ecrire en langage algorithmique (ou en C), l'algorithme de construction d'un AVL à partir d'une liste circulaire simplement chaînée donnée.
- 5) Ecrire en langage algorithmique (ou en C), l'algorithme qui retourne l'adresse d'un élément donné dans un AVL donné. Il retourne l'adresse nulle si l'élément n'existe pas.

- 6) Décrire le principe de suppression d'un élément donné dans un AVL donné.
- 7) Appliquer votre méthode à la suppression de l'élément 12 dans l'AVL ci-contre.



Exercice 4 : tas-max (6.0Pts=0.5+1.5+0.5+1.0+1.0+0.5+1.0)

Rappel : Une file de priorité (nommé Tas) permet de gérer un ensemble d'éléments selon une clé appelée leur priorité. Dans cet exercice, on confond chaque élément avec sa priorité.

- 1) Ecrire en langage C, la structure de données représentant un tas-max. (Justifier chaque membre de la structure).
- 2) Soit le tas-max suivant représenté sous forme d'un tableau :

63	34	55	27	30	49	50	13	19	29
----	----	----	----	----	----	----	----	----	----

 - a) Représenter graphiquement sous forme d'arbre binaire le tas-max ci-dessus.
 - b) Décrire graphiquement étape par étape l'insertion de l'élément 50.
 - c) Décrire graphiquement étape par étape la suppression du maximum à partir du tas défini en (a).
- 3) Etablir la relation entre la hauteur de l'arbre binaire et le nombre d'éléments présents dans le tas-max (Justifier votre réponse).
- 4) Ecrire l'algorithme d'insertion d'un élément donné dans un tas-max donné.
- 5) Justifier la propriété suivante : la complexité dans le pire des cas de l'opération d'insertion d'un élément dans un tas-max est de l'ordre $\log_2(n)$, ou n est le nombre d'éléments présent dans le tas.
- 6) Le tableau d'entiers ci-dessous est-il un Tax-Max (Justifier votre réponse) ?

9	13	57	27	31	49	50	34	29	63
---	----	----	----	----	----	----	----	----	----

- 7) Ecrire une fonction qui vérifie si un tableau donné composé de n éléments est un Tas-Max.

NOTES

- ♦ Les réponses aux questions doivent être claires et justifiées.
- ♦ Pour chaque algorithme (ou fonction en C), vous aurez soin d'expliquer les différentes actions de l'algorithme ainsi que le passage des paramètres.
- ♦ Chaque algorithme (ou fonction en C) supplémentaire utilisé doit être défini.

Annexes**On suppose connu les algorithmes suivants :****Structure Liste simplement chaînée :**

Fonction donnee : Liste $\rightarrow$ TElement
 Fonction suivant : Liste $\rightarrow$ Liste
 Fonction initL : [] $\rightarrow$ Liste
 Fonction estVideL : Liste $\rightarrow$ Booléenne

Structure Liste doublement chaînée :

Fonction donnee : DListe $\rightarrow$ TElement
 Fonction suivant : DListe $\rightarrow$ DListe
 Fonction precedent : DListe $\rightarrow$ DListe
 Fonction initDL : [] $\rightarrow$ DListe
 Fonction estVideDL : DListe $\rightarrow$ Booléenne
 Fonction initLTQ : [] $\rightarrow$ ListeTQ
 Fonction estVideLTQ : ListeTQ $\rightarrow$ Booléenne
 Procédure suppListeTQ : ListeTQ $\rightarrow$ []

Structure AVL

Fonction donnee : AVL $\rightarrow$ TElement
 Fonction hauteur : AVL $\rightarrow$ Entier
 Fonction filsGauche : AVL $\rightarrow$ AVL
 Fonction filsDroit : AVL $\rightarrow$ AVL
 Fonction initA : [] $\rightarrow$ AVL
 Fonction estVideA : AVL $\rightarrow$ Booléenne
 Fonction equilibrage : AVL $\rightarrow$ AVL

Structure File

Fonction sommetF : File $\rightarrow$ TElement
 Fonction enfiler : TElement x File $\rightarrow$ File
 Fonction defiler : File $\rightarrow$ File
 Fonction initF : [] $\rightarrow$ File
 Fonction estVideF : File $\rightarrow$ Booléenne
 Fonction estPleinF : File $\rightarrow$ Booléenne

Structure Pile

Fonction sommetP : Pile $\rightarrow$ TElement
 Fonction empiler : TElement x Pile $\rightarrow$ Pile
 Fonction depiler : Pile $\rightarrow$ Pile
 Fonction initP : [] $\rightarrow$ Pile
 Fonction estVideP : Pile $\rightarrow$ Booléenne
 Fonction estPleinP : Pile $\rightarrow$ Booléenne